Wartość i tożsamość obiektów

Po przykładach problemów ze współdzieleniem danych i ich rozwiązania przez kopiowanie można odnieść wrażenie, że obiekt przekazany do konstruktora zawsze należy skopiować. Takiej zasady jednak nie ma. Zależy to od tego, co ten obiekt reprezentuje i jakie powiązania chcemy zachować.

Pomocne jest rozróżnienie:

- **bytów o własnej tożsamości** (ang. *entities*), np. osób — interesuje nas, czy chodzi o tę samą osobę, nawet jeżeli zmieniła swoje dane;
- **obiektów reprezentujących wartość** (ang. *value objects*), np. dat — interesuje nas przede wszystkim, jaki dzień reprezentują.

Nie są to osobne rodzaje typów w Pythonie. To sposób rozumienia roli obiektów w programie.

Zmiana danych osobowych

Załóżmy na początek, że w naszym prostym programie jedną osobę reprezentuje jeden obiekt `Osoba`. Dzięki temu pytanie o tę samą osobę możemy powiązać z pytaniem o ten sam obiekt Pythona. Później zobaczymy ograniczenie tego założenia.

In [1]:
```
class Osoba:
    def __init__(self, imie, nazwisko):
        self.imie = imie
        self.nazwisko = nazwisko

    def __repr__(self):
        return f"Osoba({self.imie!r}, {self.nazwisko!r})"
```

In [2]:
```
jan = Osoba("Jan", "Kowalski")
wlasciciel = jan
```

Operator `is` sprawdza **tożsamość obiektów**, a nie zgodność wartości ich pól:

In [3]:
```
wlasciciel is jan
```

Out[3]:
```
True
```

Funkcja `id` zwraca liczbę identyfikującą obiekt Pythona przez czas jego istnienia. Dwie zmienne wskazujące ten sam obiekt dadzą ten sam wynik. Konkretna liczba może być inna przy kolejnym uruchomieniu.

In [4]:
```
id(jan)
```

Out[4]:
```
124615744295264
```

In [5]:
```
id(wlasciciel) == id(jan)
```

Out[5]:
```
True
```

In [6]:
```
jan.imie = "Piotr"
```

In [7]:
```
wlasciciel
```

Out[7]:
```
Osoba('Piotr', 'Kowalski')
```

In [8]:
```
wlasciciel is jan
```

Out[8]:
```
True
```

Zmieniło się imię, ale nadal chodzi o tę samą osobę i ten sam obiekt. Nazwa zmiennej `jan` nie zmienia się automatycznie wraz z polem `imie`.

Dwie różne osoby mogą natomiast mieć takie samo imię i nazwisko:

In [9]:
```
inna_osoba = Osoba("Piotr", "Kowalski")
```

In [10]:
```
inna_osoba.imie == jan.imie and inna_osoba.nazwisko == jan.nazwisko
```

Out[10]:
```
True
```

In [11]:
```
inna_osoba is jan
```

Out[11]:
```
False
```

Zgodność danych nie oznacza tu tożsamości. W przyjętym prostym modelu `is` dobrze odpowiada pytaniu, czy dwie zmienne dotyczą tej samej osoby.

Czy konstruktor musi zawsze kopiować przekazane obiekty?

Wróćmy do Jana Kowalskiego i utwórzmy dla niego konto. Konto ma zapamiętać swojego właściciela. **Z powodu utworzenia konta nie powstaje nowy Jan Kowalski — jako kopia starego.**

In [12]:
```
class Konto:
    def __init__(self, wlasciciel):
        self.wlasciciel = wlasciciel
```

In [13]:
```
jan = Osoba("Jan", "Kowalski")
```

In [14]:
```
konto = Konto(jan)
```

In [15]:
```
konto.wlasciciel is jan
```

Out[15]:
```
True
```

In [16]:
```
jan.imie = "Piotr"
```

In [17]:
```
konto.wlasciciel.imie
```

Out[17]:
```
'Piotr'
```

O to nam chodziło: osoba zmieniła imię i konto nadal wskazuje jej aktualne dane.

Gdyby konstruktor zapamiętał kopię właściciela, zmiana oryginału nie byłaby widoczna na koncie. Sprawdźmy ten wariant:

In [18]:
```
import copy
```

In [19]:
```
class KontoZKopia:
    def __init__(self, wlasciciel):
        self.wlasciciel = copy.copy(wlasciciel)
```

In [20]:
```
jan = Osoba("Jan", "Kowalski")
konto_z_kopia = KontoZKopia(jan)
```

In [21]:
```
jan.imie = "Piotr"
```

In [22]:
```
konto_z_kopia.wlasciciel.imie
```

Out[22]:
```
'Jan'
```

In [23]:
```
konto_z_kopia.wlasciciel is jan
```

Out[23]:
```
False
```

Kopia ma nadal imię Jan. Nie chcieliśmy jednak osobnego, niezależnie edytowanego właściciela, lecz odniesienia do istniejącej osoby. Kopiowanie zepsuło potrzebne nam powiązanie.

Dwie daty o tej samej wartości

Inaczej zwykle traktujemy daty. Dwa obiekty opisujące ten sam dzień są dla nas równoważne: nie interesuje nas, który egzemplarz daty trafił do programu.

Użyjemy typu `date`, który ma już zdefiniowane porównywanie wartości. Operator `==` sprawdza równość według reguł danego typu, a `is` — nadal tożsamość obiektów.

In [24]:
```
from datetime import date
```

In [25]:
```
pierwsza_data = date(2026, 10, 31)
druga_data = date(2026, 10, 31)
```

In [26]:
```
pierwsza_data == druga_data
```

Out[26]:
```
True
```

In [27]:
```
pierwsza_data is druga_data
```

Out[27]:
```
False
```

In [28]:
```
id(pierwsza_data) == id(druga_data)
```

Out[28]:
```
False
```

Daty mają jednakową wartość, chociaż są różnymi obiektami Pythona. Zwykle wystarczy nam tu odpowiedź na pytanie „jaki to dzień?”.

Podobnie możemy traktować czas trwania, parę współrzędnych czy kwotę wraz z walutą. To przykłady **obiektów wartości**. Dla nich istotna jest wartość, a nie tożsamość konkretnego obiektu.

To, że klasę chcemy traktować w ten sposób, nie zmienia samo z siebie działania `==`. We własnej klasie trzeba określić reguły porównywania (służąca do tego metoda `__eq__` będzie omówiona dalej przy omawianiu metod specjalnych). Nie używamy też `is` do porównywania wartości liczb ani napisów.

Niemutowalne obiekty wartości

Jeżeli ważna jest tylko wartość, często wygodnie jest jej nie zmieniać wewnątrz istniejącego obiektu. Operacja może zamiast tego zwrócić obiekt reprezentujący nową wartość.

Tak działa niemutowalna data typu `date` z modułu `datetime`. Dwa obiekty mogą korzystać z tej samego obiektu daty bez ryzyka, że jeden zmieni współdzielone dane drugiemu. W razie potrzeby zmiany daty, trzeba utworzyć nowy obiekt — nie da się zmienić współdzielonego obiektu.

In [ ]:
```
egzamin_jana = date(2026, 10, 31)
egzamin_anny = egzamin_jana
```

In [ ]:
```
egzamin_anny = egzamin_anny.replace(day=30)
```

In [ ]:
```
egzamin_jana
```

Out[ ]:
```
datetime.date(2026, 10, 31)
```

In [ ]:
```
egzamin_anny
```

Out[ ]:
```
datetime.date(2026, 10, 30)
```

Metoda `replace` zwróciła datę z innym dniem. Przypisanie zmieniło to, na jaki obiekt wskazuje `egzamin_anny`, ale obiekt wskazywany przez `egzamin_jana` pozostał bez zmian. Nie trzeba było wcześniej robić kopii na zapas.

Takie podejście ogranicza problemy ze współdzieleniem: raz utworzoną wartość możemy przekazywać dalej bez obawy, że jej stan zostanie przypadkowo zmieniony.

Nie wystarczy jednak usunąć z własnej klasy metod zmieniających pola, żeby jej obiekty stały się niemutowalne. Jeśli nadal można przypisać nową wartość do publicznego pola, stan można zmienić. Do kontrolowania dostępu do pól wrócimy przy omawianiu tematu enkapsulacji.

Reguły czy wskazówki?

Opisane powyżej podejście do kopiowania w zależności od roli obiektu nie jest żelazną regułą, a raczej wskazówką projektową.

Różne wartości — ten sam obiekt

Trasę możemy rozumieć jako wartość określoną przez jej przebieg. Przesunięcie punktu oznacza wtedy zmianę wartości trasy, a w tym sensie — powstanie nowej trasy.

Nie narzuca to jednak, żeby program przy każdym przesunięciu tworzył od nowa całą wynikową listę. W edytorze wygodne może być modyfikowanie dotychczasowej listy punktów. Jeżeli inne miejsca programu nie potrzebują poprzedniej wersji, takie rozwiązanie może być odpowiednie.

Gdy chcemy zachować historię wszystkich wersji, sensowne może być tworzenie nowej wersji zamiast zmieniania starej. Musimy wtedy także zadbać, żeby późniejsza edycja punktów nowej wersji nie zmieniała starych. Możemy użyć kopii mutowalnych danych albo zaprojektować reprezentację niemutowalną.

Pokazana wcześniej implementacja tras używała niezależnych kopii dla każdego użytkownika. Inny serwis mógłby przechowywać wspólny katalog tras, a dodanie do ulubionych oznaczałoby odniesienie do istniejącej trasy. Wybór zależy od potrzeb programu.

Wspólna tożsamość — różne obiekty

Na początku założyliśmy jeden obiekt Pythona dla jednej osoby. W większym programie to założenie nie musi być spełnione. Program może odczytać dane tej samej osoby dwa razy, z różnych plików i w różnych miejscach.

Wtedy powstaną dwa obiekty i operator `is` zwróci `False`, mimo że oba dotyczą tej samej osoby. Same imię i nazwisko nie wystarczą do jej rozpoznania. Potrzebny może być jednoznaczny identyfikator, który skojarzony jest z daną osobą we wszystkich źródłach danych.

In [33]:
```
class OsobaZId:
    def __init__(self, identyfikator, imie, nazwisko):
        self.identyfikator = identyfikator
        self.imie = imie
        self.nazwisko = nazwisko
```

Zasymulujmy dwa odczyty danych z różnych plików:

In [34]:
```
pierwszy_odczyt = OsobaZId(17, "Piotr", "Kowalski")
drugi_odczyt = OsobaZId(17, "Piotr", "Kowalski")
```

In [35]:
```
pierwszy_odczyt is drugi_odczyt
```

Out[35]:
```
False
```

In [36]:
```
pierwszy_odczyt.identyfikator == drugi_odczyt.identyfikator
```

Out[36]:
```
True
```

Przy założeniu, że nasz system przydziela jednej osobie zawsze ten sam identyfikator, porównanie identyfikatorów pozwala rozpoznać tę samą osobę mimo różnych obiektów Pythona.

Tutaj rozdzielona jest już ściśle **tożsamość obiektu Pythona** i **tożsamość osoby w modelowanym świecie**. Wartość zwracana przez funkcję `id` opisuje tę pierwszą, ale nie jest ani trwałym ani jednoznacznym identyfikatorem osoby i nie należy np. zapisywać jej w pliku w tej roli.

Dwa odrębne obiekty mogą też z czasem zawierać różne dane tej samej osoby. Wspólny identyfikator pozwala ją rozpoznać, ale sam nie synchronizuje jej danych. To dodatkowy problem projektowy, którego nie rozwiązuje ani użycie operatora `is`, ani użycie funkcji `deepcopy`.

Kiedy kopiować?

Nie kopiujemy obiektu tylko dlatego, że przekazano go do konstruktora. Musimy rozważyć, czy potrzebujemy niezależnie zmienianych danych, czy powiązania z istniejącym obiektem.

- Dla właściciela konta chcieliśmy zachować powiązanie z osobą i widzieć jej zmiany.
- Dla prywatnej kopii trasy chcieliśmy móc edytować punkty bez wpływu na oryginał.
- Dla niemutowalnej daty samo współdzielenie nie grozi zmianą danych, więc kopiowanie zwykle jest zbędne.

To rozróżnienie pomaga zbudować intuicję, ale sposób jego realizacji musimy dopasować do konkretnego programu.